Ingeniería de Software Asistida por IA

Tema 6: Calidad de Código, TDD y Deuda Cero

Guía de Estudio Detallada para Estudiantes

El Mito Peligroso: "Si funciona, no lo leo"

Existe un mito generalizado entre los desarrolladores novatos de IA: *"Si la Inteligencia Artificial escribió el código, me generó los tests y todo está en verde, entonces no necesito leer el código resultante"*.

Este enfoque garantiza el fracaso del proyecto a mediano plazo. Las IAs suelen dejar "Bugs Fantasma" (variables redundantes, ciclos anidados innecesarios, acoplamiento fuerte) que no rompen la aplicación hoy, pero la hacen inmodificable mañana. El objetivo de leer el código generado no es encontrar errores de sintaxis (el compilador hace eso), es **desarrollar un Ojo Crítico**. Al leer las malas decisiones de la IA, aprendes a modificar tu *prompt* futuro para prohibirle volver a tomar ese camino.

TDD Asistido (Test-Driven Development)

El Desarrollo Guiado por Pruebas (TDD) es la correa perfecta para controlar matemáticamente a la máquina. Al invertir el orden tradicional de desarrollo, obligas a la IA a ceñirse al contrato sin margen de creatividad.

Flujo de TDD con IA: 1. Defines la Interfaz o Contrato (Entradas y Salidas). 2. Pides a la IA que **solo escriba la Prueba Unitaria** (aserciones lógicas) contemplando los casos límite. 3. **Auditoría de Test:** Tú lees la prueba y decides si la lógica de negocio tiene sentido. 4. Pides a la IA que escriba el código para que esa prueba pase. 5. **Auditoría de Código:** Aunque pase la prueba, revisas que el código no sea espagueti.

El Principio de Responsabilidad Única (SRP)

Las IAs tienden a condensar la lógica. Una función que la IA te entregue probablemente validará los datos de un formulario, calculará los impuestos, guardará en la base de datos y enviará un correo electrónico. Esto viola el **Principio de Responsabilidad Única** (Una función debe hacer una sola cosa).

El Rol del Bombero: Cuando detectas esto, tu trabajo es rechazar la entrega. Ordenas a la IA: *"Este bloque funciona, pero es código basura acoplado. Refactorízalo dividiéndolo en 4 funciones puras e independientes"*. Reducir la Deuda Técnica a cero con IA toma segundos; aceptarla por pereza es injustificable.

Desarrollo Orientado a Bibliotecas

La máxima prueba de fuego de la calidad del código es su **Reutilización**. Si el botón de procesamiento de pagos que hizo la IA en tu aplicación solo sirve para esta aplicación (porque tiene URLs, variables de estado o lógica de negocio *hardcodeada* por dentro), es un código mediocre.

El objetivo del Arquitecto es extraer componentes agnósticos ("Piezas de Lego"). Si le exiges a la IA que use **Inyección de Dependencias** (pasar las funciones complejas como parámetros), convertirás ese código mediocre en una Biblioteca genérica que podrás llevar a tu próximo proyecto sin tener que reescribir nada.

La Evolución Final: Orquestación en Paralelo

Hasta ahora te enseñamos a trabajar un archivo a la vez ("Sprints Estancos") para evitar que la IA alucinara. Esas eran las "rueditas de entrenamiento". El salto real de productividad ocurre al paralelizar.

Dado que en el Taller 3 aprendiste a definir **Contratos Duros (Interfaces)**, ahora posees una frontera de confianza absoluta. Esto significa que puedes abrir dos chats diferentes: a un agente (ej. Claude) le ordenas crear el servicio de Backend, y a otro (ej. ChatGPT) le ordenas crear la UI de React **al mismo tiempo**.

Como les pasas el mismo Contrato estricto a ambos, tienes garantía matemática de que el código encajará cuando lo unas. Has dejado de ser un programador asistido para convertirte en un **Project Manager Multi-Agente**.

El Code Review Final: Las 6 Dimensiones

Al finalizar un proyecto asistido por IA, este no se evalúa por si "la app prende". Se somete a una auditoría estricta de 6 dimensiones: 1. **Trazabilidad:** ¿Existen "Cajas Mágicas" que nadie sabe cómo funcionan? 2. **Deuda Técnica:** ¿Existen comentarios pendientes o código duplicado? 3. **Modularidad:** ¿Los componentes pueden ser extraídos a otro proyecto? 4. **Calidad de Código:** ¿Las variables tienen nombres descriptivos? ¿No se usa any? 5. **Documentación:** ¿El `AGENTS.md` y los contratos reflejan la realidad del código? 6. **Intercambiabilidad de Roles:** La prueba final. Cualquier miembro del grupo debe poder justificar lógicamente el código que la IA generó para los demás compañeros. Si no puedes defender el código, no es tuyo.